MODULE 22 · DOMAIN 4 - ASSEMBLING AND DEPLOYING APPLICATIONS

Sizing AI Search for a workload

Standard or storage-optimised, and the levers that trade latency, throughput, freshness and cost.

SECTION 4 of 6 EXAM WEIGHT 22% MODULE 22 of 40

admin@sthithapragna.com

A sports car or a lorry? Standard endpoints are fast and nimble. Storage-optimised endpoints carry far more, far more cheaply, a little more slowly.

Databricks' performance guidelines

ENDPOINT TYPE	LATENCY	QUERIES PER SECOND	CAPACITY	ONE UNIT HOLDS
Standard	20 to 50 ms	30 to 200+	320 million vectors	2 million vectors
Storage-optimised	300 to 500 ms	30 to 50	1 billion vectors	64 million vectors

ON THE EXAM

Storage-optimised is up to 7 times cheaper per vector and indexes 10 to 20 times faster. It pays for that in latency.

Standard, or storage-optimised?

STANDARD WHEN

Latency is critical

- Index well under 320 million vectors
- Tens of milliseconds needed
- High or bursty query rates
- Continuous sync or target QPS needed

STORAGE-OPTIMISED WHEN

Scale and cost matter most

- 10 million vectors or more
- Extra latency is acceptable
- Cost per vector matters
- Fast re-indexing matters

THE TIEBREAKER

Databricks' own rule: standard when latency is critical and the index is well under 320 million. Storage-optimised from about 10 million, if latency can give.

What happens as an index grows

- Performance is best when the index fits in **one unit**: 2 million standard, 64 million storage-optimised.
- Beyond one unit, **latency rises and throughput tapers**, towards about 30 queries per second.
- Filters, dimensions and concurrency all move the numbers.
- Halving dimensions, say 768 to 384, gives about **1.5 times the throughput** and about 20% lower latency.
- Latency can degrade as traffic ramps, even when average use looks low.

> ON THE EXAM
>
> Choose the smallest dimensionality that preserves retrieval quality. It is the cheapest performance lever you have.

What each lever buys, and what it costs

LEVER	BUYS	COSTS
Storage-optimised endpoint	Scale, cheaper vectors, fast indexing	Hundreds of ms per query
Target QPS, standard only	Higher sustained throughput	Paid for even when idle
Fewer dimensions	Throughput and latency	Some quality, if cut too far
Continuous sync	Freshness in seconds	An always-on pipeline
Hybrid search and reranker	Better retrieval quality	Extra latency

ON THE EXAM

Every lever trades one of latency, throughput, freshness, quality or cost for another. Read which one the scenario can spare.

Setting a target QPS

- By default, standard endpoints handle about **20 to 200 QPS**, depending on index size.
- On **standard endpoints only**, set a **target QPS** and Databricks provisions for it.
- Best effort, not guaranteed, and **no autoscaling**: above the target you get 429 errors.
- You are **charged for the capacity** whether or not traffic arrives.
- For text queries, the **embedding model** can be the real bottleneck.

> ON THE EXAM
>
> If 429s continue with text queries, look at the embedding endpoint: a smaller model, or a dedicated one for queries.

When one endpoint is not enough

- **Split indexes** across endpoints when several busy indexes share one.
- **Replicate** one very busy index across endpoints, and split traffic in the client.
- Replication gives roughly linear gains in throughput.
- These are Databricks' suggestions when target QPS is unavailable or not enough.

> ON THE EXAM
>
> Several busy indexes: give each its own endpoint. One very busy index: copy it and spread the traffic.

The order to check things in

01
Size
How many vectors? Under or over 320 million?

02
Latency
Tens of ms, or can it give?

03
Throughput
Sustained QPS, and bursts?

04
Freshness
Seconds, or scheduled?

05
Extras
Can latency afford hybrid and reranking?

ON THE EXAM

Stop at the first hard constraint. A latency requirement in milliseconds rules out storage-optimised before cost is considered.

Pennant Outdoor sizes four indexes

WORKLOAD	CHOICE
Search bar: 5M vectors, 150 QPS, under 100 ms	Standard, target QPS, reranker off
Support chat: 2M vectors, 3 QPS, seconds allowed	Standard, triggered, reranker on
Product archive: 700M vectors, internal, 1 s fine	Storage-optimised, triggered
Reviews: rebuilt weekly, 90M vectors, batch use	Storage-optimised, fast re-indexing

ON THE EXAM

Same company, four answers. The workload decides, not a house standard.

> An internal research tool must search 800 million document chunks. Researchers are happy to wait up to a second for results.
>
> The budget is tight, and query volume is modest.

Which configuration fits best?

A. A storage-optimised endpoint, with triggered sync

B. A standard endpoint with a high target QPS

C. Three standard endpoints, each holding part of the data

D. A Direct Vector Access index on a standard endpoint

◆ PAUSE AND DECIDE

Which configuration fits best?

A Storage-optimised

Holds a billion vectors, cheapest per vector, latency acceptable.

B Standard, high target QPS

800M exceeds standard capacity, and QPS was never the issue.

C Three standard endpoints

Workable, but far costlier, and needs custom merging.

D Direct Vector Access

Changes who writes vectors, not capacity or cost.

ON THE EXAM

800 million vectors, a second is fine, tight budget. Every signal points to storage-optimised.

> A product recommendation widget queries a 5-million-vector index on a standard endpoint. It needs 120 queries per second, sustained, in under 100 milliseconds.
>
> Under normal load, requests start failing with 429 errors.

What should the engineer do first?

A. Move the index to a storage-optimised endpoint

B. Turn on the reranker to improve result quality

C. Set a target QPS on the existing standard endpoint

D. Switch the index from triggered to continuous sync

◆ PAUSE AND DECIDE

What should the engineer do first?

A Storage-optimised

Lower QPS and 300 ms or more. Fails both requirements.

B Reranker

Adds latency, and does nothing for throughput.

C Target QPS

Provisions for sustained throughput on standard endpoints.

D Continuous sync

Affects freshness, not query capacity.

THE SIGNAL

429s under normal load at 120 sustained QPS. That is exactly what a target QPS is for.

> The same widget now has a target QPS set. 429 errors continue after scaling completes.
>
> The application sends text queries to a Delta Sync index that uses managed databricks-gte-large-en embeddings.

What is the most likely remaining bottleneck, and fix?

A The endpoint type: move the index to storage-optimised

B Result quality: turn on the reranker to reduce retries

C Freshness: switch from triggered to continuous sync

D The embedding model: use a smaller or dedicated one

◆ PAUSE AND DECIDE

What is the most likely remaining bottleneck, and fix?

A Storage-optimised

Lower throughput still. And target QPS is standard-only.

B Reranker

Adds a heavier step to every query.

C Continuous sync

Nothing to do with query throughput.

D Smaller or dedicated embedding model

Text queries are embedded on every call. That step can throttle.

THE SIGNAL

Text queries plus managed embeddings, and 429s despite target QPS. Every query is embedded first, so check that model.

> An index on a standard endpoint has grown from 1.5 million to 40 million vectors at 1,024 dimensions. Latency has risen and throughput has dropped.
>
> Evaluation shows 512-dimension embeddings retrieve almost as well for this data.

Which change most directly improves latency and throughput?

A Switch the index from triggered to continuous sync

B Reduce the embedding dimensions to 512 and re-embed

C Turn on hybrid search and the reranker for every query

D Sync every metadata column into the index as well

◆ PAUSE AND DECIDE

Which change most directly improves latency and throughput?

A Continuous sync

Freshness, not query speed. Adds cost.

B Halve the dimensions

About 1.5x throughput and 20% less latency, quality kept.

C Hybrid and reranker

More work per query: slower, not faster.

D Sync all metadata

Bigger records, no speed benefit.

THE SIGNAL

Grown well beyond one unit, and evaluation says half the dimensions is nearly as good. Take the cheapest lever.

> A marketplace rebuilds its 300-million-vector listings index every weekend from a fresh export. The rebuild must finish quickly.
>
> Queries come from a nightly batch process, where half a second per query is fine. Cost matters.

Which configuration fits best?

A. A standard endpoint with continuous sync

B. A storage-optimised endpoint with triggered sync

C. A standard endpoint with a high target QPS

D. A Direct Vector Access index, rewritten weekly

◆ PAUSE AND DECIDE

Which configuration fits best?

A. Standard, continuous
Pays for always-on freshness on data that changes weekly.

B. Storage-optimised, triggered
10 to 20 times faster indexing, cheaper vectors, latency fine.

C. Standard, high target QPS
A batch job does not need high sustained QPS.

D. Direct Vector Access
Hand-written loading instead of an automatic sync.

THE SIGNAL

Rebuilt weekly, must finish quickly, batch queries, cost matters. Fast indexing plus low cost is storage-optimised.

> Two indexes, one for support articles and one for product specs, share a single standard endpoint. Both receive heavy traffic.
>
> Target QPS is not available in this workspace, and throughput is insufficient.

What does Databricks recommend?

A Merge both indexes into one larger index

B Move both indexes to a storage-optimised endpoint

C Turn on the reranker to make each query count

D Host each of the two indexes on its own endpoint

◆ PAUSE AND DECIDE

What does Databricks recommend?

A **Merge the indexes**
One bigger index on the same endpoint: slower, not faster.

B **Storage-optimised**
Lower throughput per endpoint. Worse.

C **Reranker**
Adds latency, adds no capacity.

D **Each index on its own endpoint**
Stops them competing, raising total throughput.

THE SIGNAL

Several busy indexes on one endpoint, and no target QPS. Split them across endpoints.

> A voice assistant must fetch context in well under 100 milliseconds before speaking. Its index holds 60 million vectors.
>
> Traffic is about 40 queries per second. A colleague notes that storage-optimised is recommended above 10 million vectors.

Which endpoint type should the team choose?

A Standard, because latency is the critical constraint

B Storage-optimised, because it has over 10 million vectors

C Storage-optimised, with a target QPS added

D Either, because both handle 40 queries a second

◆ PAUSE AND DECIDE

Which endpoint type should the team choose?

A Standard

Tens of ms, and 60M is well under 320M.

B Storage-optimised, over 10M

The 10M guide assumes latency can give. Here it cannot.

C Storage-optimised plus target QPS

Not possible: target QPS is standard-only.

D Either

Throughput fits both. Latency fits only one.

THE SIGNAL

Well under 100 milliseconds is a hard constraint. It rules out storage-optimised before size or cost are weighed.

> An internal policy index holds 3 million vectors, updated nightly, and receives about 2 queries per second.
>
> It was set up with continuous sync and a target QPS of 300, just in case. Finance wants the cost cut.

What should the engineer change?

A Move it to a storage-optimised endpoint to cut costs

B Turn on the reranker to reduce repeat queries

C Remove the target QPS, and switch to triggered sync

D Replicate the index across two standard endpoints

◆ PAUSE AND DECIDE

What should the engineer change?

A Storage-optimised

Built for 10M+ vectors. The waste here is elsewhere.

B Reranker

Adds cost and latency.

C No target QPS, triggered sync

Stops paying for idle capacity and an always-on pipeline.

D Replicate the index

Doubles cost for throughput nobody needs.

THE SIGNAL

Target QPS is charged whether traffic arrives or not, and continuous sync runs all day for nightly data. Remove both.

What to carry forward

- Standard: **20 to 50 ms**, up to 320M vectors. Storage-optimised: **300 to 500 ms**, up to 1B, up to 7x cheaper.
- Latency critical and well under 320M: standard. 10M+ and latency can give: storage-optimised.
- **Target QPS**: standard only, charged even when idle, 429s above it.
- Halve dimensions for about 1.5x throughput. Check the embedding model for text queries.
- Busy indexes: split across endpoints, or replicate one. Read size, latency, QPS, freshness, extras.

ON THE EXAM

Next: module 23 packages a chain with pyfunc, signatures and Unity Catalog.